

MULTI LEVEL PARKING LOT MANAGER

Project Team

Gangannagari Manith REDDY – 2620040072

Kamineni Jathin – 2620030527

Faculty Guide: Gopi Krishna

1. Introduction

A Multi Level Parking Lot Manager is a Java-based console application for organizing vehicles across multiple parking floors.

It keeps track of parking slots, vehicle details, entry time and exit information.

The system automatically assigns the first available slot and displays the current parking status.

It demonstrates core Java concepts such as classes, objects, arrays, collections, methods and conditional statements.

2. Problem Statement

Manual parking management becomes difficult when the parking area has multiple floors and many vehicles.

Finding an empty slot can take time.

Paper-based records can be difficult to update and search.

Vehicle entry and exit details may be missed or recorded incorrectly.

Parking fee calculation can also be time-consuming when done manually.

3. Objectives

Manage multiple parking floors and parking slots.

Automatically allocate an available parking slot.

Store vehicle number and vehicle type.

Display available and occupied parking spaces.

Search for a parked vehicle.

Remove a vehicle and calculate its parking fee.

Provide a simple and user-friendly Java console interface.

4. Existing System

Parking is often managed using manual registers or basic supervision.

Staff may need to check each floor to find available spaces.

Vehicle records can be maintained on paper.

Searching for a particular vehicle may take additional time.

Manual fee calculation can lead to calculation mistakes.

5. Proposed System

The proposed system is a Java console application that manages a three-floor parking lot.

Each floor contains multiple parking slots.

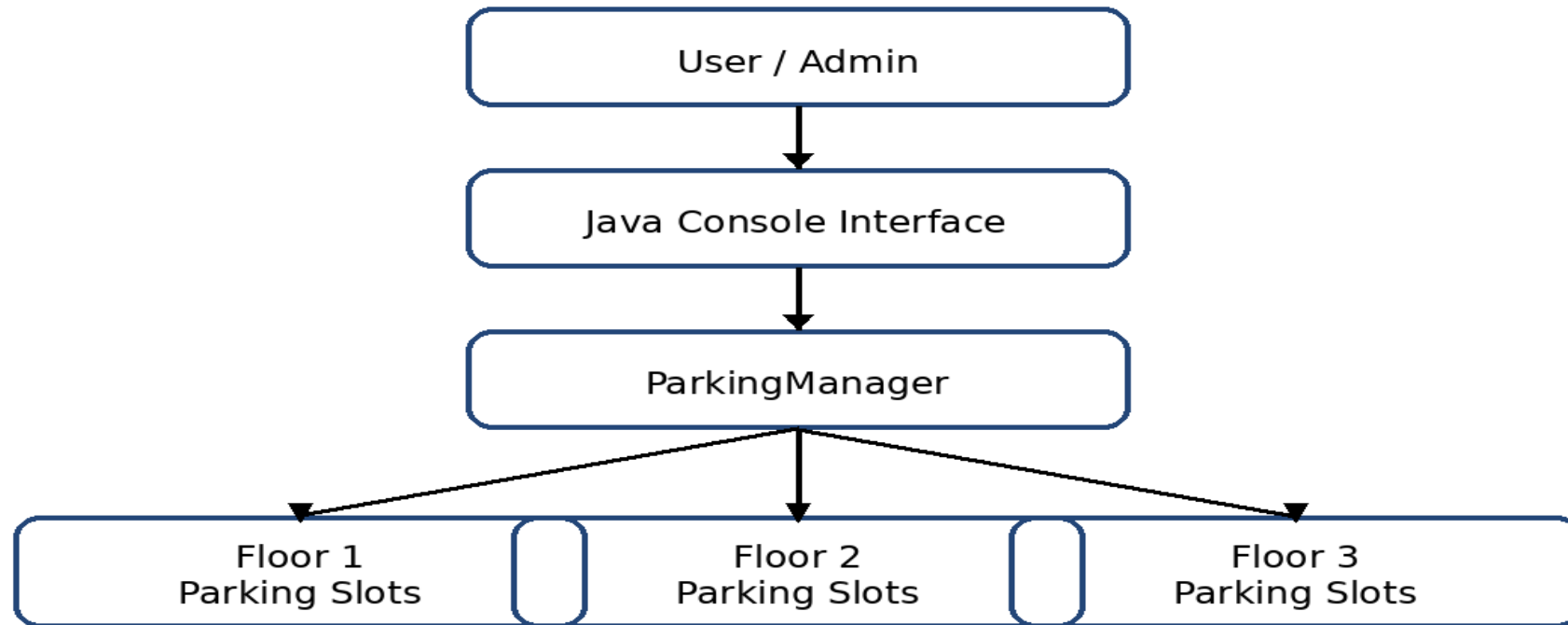
When a vehicle enters, the system checks floors and assigns the first available slot.

When a vehicle exits, the slot becomes available again.

The system calculates parking charges according to vehicle type and parking duration.

6. System Architecture

Multi Level Parking Lot Manager - Architecture



7. Functionalities

1. Park Vehicle — records the vehicle and assigns a free slot.
2. Remove Vehicle — removes the vehicle and calculates the fee.
3. Display Parking Status — shows all floors and slot status.
4. Search Vehicle — finds the floor and slot of a vehicle.
5. Exit — closes the application.

Vehicle types supported: Bike, Auto and Car.

8. Implementation – Java Code Screenshots

ParkingManager.java - Vehicle Parking & Allocation

```
import java.time.Duration;
import java.time.LocalDateTime;
import java.util.ArrayList;

public class ParkingManager {
    private ArrayList<ParkingFloor> floors;

    public ParkingManager(int numberOfFloors, int slotsPerFloor) {
        floors = new ArrayList<>();

        for (int i = 1; i <= numberOfFloors; i++) {
            floors.add(new ParkingFloor("Floor " + i, slotsPerFloor));
        }
    }

    public boolean parkVehicle(String number, String type) {
        if (findVehicle(number) != null) {
            System.out.println("Vehicle is already parked.");
            return false;
        }

        for (ParkingFloor floor : floors) {
            ParkingSlot slot = floor.findAvailableSlot();

            if (slot != null) {
                Vehicle vehicle = new Vehicle(number, type);
                vehicle.setParkingLocation(floor.getFloorName(), slot.getSlotNumber());
                slot.parkVehicle(vehicle);

                System.out.println("\nVehicle parked successfully.");
                System.out.println("Vehicle : " + number);
                System.out.println("Floor   : " + floor.getFloorName());
                System.out.println("Slot    : " + slot.getSlotNumber());
                return true;
            }
        }

        System.out.println("Parking lot is full.");
        return false;
    }

    public void removeVehicle(String number) {
        for (ParkingFloor floor : floors) {
            ParkingSlot slot = floor.findVehicleSlot(number);

            if (slot != null) {
                Vehicle vehicle = slot.getVehicle();
                long hours = getParkingHours(vehicle.getEntryTime());
                double fee = calculateFee(vehicle.getType(), hours);

                slot.removeVehicle();

                System.out.println("\nVehicle removed successfully.");
                System.out.println("Vehicle : " + number);
                System.out.println("Duration: " + hours + " hour(s)");
                System.out.println("Fee      : Rs." + fee);
                return;
            }
        }

        System.out.println("Vehicle not found.");
    }
}
```

Main.java - Menu and User Input

```
import java.util.Scanner;

public class Main {
    public static void main(String[] args) {
        Scanner sc = new Scanner(System.in);
        ParkingManager manager = new ParkingManager(3, 5);

        int choice;

        do {
            System.out.println("\n=====");
            System.out.println("  MULTI LEVEL PARKING LOT MANAGER");
            System.out.println("=====");
            System.out.println("1. Park Vehicle");
            System.out.println("2. Remove Vehicle");
            System.out.println("3. Display Parking Status");
            System.out.println("4. Search Vehicle");
            System.out.println("5. Exit");
            System.out.print("Enter your choice: ");
            choice = sc.nextInt();

            switch (choice) {
                case 1:
                    System.out.print("Enter vehicle number: ");
                    String number = sc.next();

                    System.out.print("Enter vehicle type (bike/auto/car): ");
                    String type = sc.next();

                    manager.parkVehicle(number, type);
                    break;

                case 2:
                    System.out.print("Enter vehicle number: ");
                    manager.removeVehicle(sc.next());
                    break;

                case 3:
                    manager.displayStatus();
                    break;

                case 4:
                    System.out.print("Enter vehicle number: ");
                    manager.searchVehicle(sc.next());
                    break;

                case 5:
                    System.out.println("Thank you for using the Parking Manager.");
                    break;

                default:
                    System.out.println("Invalid choice.");
            }
        } while (choice != 5);

        sc.close();
    }
}
```

9. Output Screenshots

Sample Program Output

```
=====
MULTI LEVEL PARKING LOT MANAGER
=====
1. Park Vehicle
2. Remove Vehicle
3. Display Parking Status
4. Search Vehicle
5. Exit
Enter your choice: Enter vehicle number: Enter vehicle type (bike/auto/car):
Vehicle parked successfully.
Vehicle : TS09AB1234
Floor : Floor 1
Slot : 1

=====
MULTI LEVEL PARKING LOT MANAGER
=====
1. Park Vehicle
2. Remove Vehicle
3. Display Parking Status
4. Search Vehicle
5. Exit
Enter your choice: Enter vehicle number: Enter vehicle type (bike/auto/car):
Vehicle parked successfully.
Vehicle : TS09CD5678
Floor : Floor 1
Slot : 2

=====
MULTI LEVEL PARKING LOT MANAGER
=====
1. Park Vehicle
2. Remove Vehicle
3. Display Parking Status
4. Search Vehicle
5. Exit
Enter your choice:
===== PARKING STATUS =====

Floor 1 :
Slot 1 : TS09AB1234 (car)
Slot 2 : TS09CD5678 (bike)
Slot 3 : Available
Slot 4 : Available
Slot 5 : Available

Floor 2 :
Slot 1 : Available
Slot 2 : Available
Slot 3 : Available
Slot 4 : Available
Slot 5 : Available

Floor 3 :
Slot 1 : Available
Slot 2 : Available
Slot 3 : Available
Slot 4 : Available
Slot 5 : Available

=====
MULTI LEVEL PARKING LOT MANAGER
=====
1. Park Vehicle
2. Remove Vehicle
3. Display Parking Status
4. Search Vehicle
5. Exit
Enter your choice: Enter vehicle number:
Vehicle found.
Vehicle : TS09AB1234
Type : car
Floor : Floor 1
Slot : 1

=====
MULTI LEVEL PARKING LOT MANAGER
=====
1. Park Vehicle
2. Remove Vehicle
3. Display Parking Status
4. Search Vehicle
5. Exit
Enter your choice: Enter vehicle number:
Vehicle removed successfully.
Vehicle : TS09AB1234
Duration: 1 hour(s)
Fee : Rs.30.0

=====
MULTI LEVEL PARKING LOT MANAGER
=====
1. Park Vehicle
2. Remove Vehicle
3. Display Parking Status
4. Search Vehicle
5. Exit
Enter your choice: Thank you for using the Parking Manager.
```

10. Future Scope

Develop a graphical user interface (GUI).

Connect the application to a database for permanent records.

Add online parking reservation.

Integrate QR-code based entry and exit.

Support digital payments.

Add real-time parking availability and mobile application support.

Integrate automatic number-plate recognition in a future version.

11. Conclusion

The Multi Level Parking Lot Manager provides an organized way to manage parking spaces and vehicles.

It reduces the effort required to find available slots and maintain vehicle records.

The project implements important Java programming concepts in a practical application.

The system can be extended with databases, GUI, online booking and digital payment features.

THANK YOU

Gangannagari Manith REDDY – 2620040072

Kamineni Jathin – 2620030527

Faculty Guide: Gopi Krishna